CS 3600: Introduction to Artificial Intelligence

Lecture Notes

Aaron Hillegass — Georgia Tech

Contents

1 Probability for AI **4**
1.1 Motivation . . . 4
1.2 Probability Basics . . . 4
1.3 Joint, Marginal, and Conditional Probability . . . 4
1.4 Chain Rule for Probability . . . 4
1.5 Independence . . . 4
1.6 Bayes' Law . . . 5
1.7 Continuous Random Variables . . . 5
1.8 Working in Log Space . . . 5

2 Bayes' Rule with Continuous Random Variables **5**
2.1 Continuous Bayes . . . 5
2.2 Exponential Distribution . . . 6
2.3 Credible Intervals . . . 6
2.4 Three Approaches to Continuous Posteriors . . . 6
2.5 Binomial Example: Polling . . . 6

3 Bayesian Networks **7**
3.1 Motivation: Modeling Dependencies . . . 7
3.2 Structure of a Bayesian Network . . . 7
3.3 Conditional Independence . . . 7
3.4 Information Flow: Three Structures . . . 7
3.5 Inference in Bayes Nets . . . 7
3.6 Continuous Variables in Bayes Nets . . . 7

4 Bayesian and Naive Bayes Classifiers **8**
4.1 Classification via Bayes' Rule . . . 8
4.2 Naive Bayes Classifier . . . 8
4.3 Multivariate Gaussian Classifier . . . 8
4.4 Conditional Distribution of a Multivariate Gaussian . . . 8
4.5 Parameterless Density Estimation . . . 8
4.6 Summary of Classifier Types . . . 9

5 Markov Chains and Hidden Markov Models 9
5.1 Markov Chains . . . 9
5.2 Mini-Forward Algorithm . . . 9
5.3 Stationary Distribution . . . 9
5.4 Hidden Markov Models (HMMs) . . . 9
5.5 Key HMM Algorithms . . . 10
5.6 Application: Speech Recognition . . . 10
5.7 Google PageRank . . . 10

6 Kalman Filters and Particle Filters 10
6.1 The Localization Problem . . . 10
6.2 Particle Filters . . . 10
6.3 Kalman Filters . . . 11
6.4 Kalman Filter Loop . . . 11

7 Intro to ML, K-Nearest Neighbors, and Decision Trees 12
7.1 Flavors of Machine Learning . . . 12
7.2 ML Workflow . . . 12
7.3 K-Nearest Neighbors (kNN) . . . 12
7.4 Validation and Cross-Validation . . . 12
7.5 Common Problems and Fixes . . . 13
7.6 Decision Trees . . . 13
7.7 Ensemble Methods . . . 13

8 Linear Regression and Gradient Descent 13
8.1 The Machine Learning Workflow . . . 13
8.2 Simple Linear Regression . . . 14
8.2.1 The Model . . . 14
8.2.2 The Bias-Variance Tradeoff . . . 14
8.3 Loss Functions . . . 14
8.3.1 L1 Loss (Mean Absolute Error) . . . 14
8.3.2 L2 Loss (Mean Squared Error) . . . 14
8.4 Solving Linear Regression with Matrices . . . 14
8.5 Gradient Descent . . . 15
8.5.1 The Gradient . . . 15
8.5.2 The Algorithm . . . 15
8.5.3 Choosing the Learning Rate . . . 15
8.5.4 Local Minima in High Dimensions . . . 15
8.6 Practical Improvements to Gradient Descent . . . 15
8.6.1 Data Standardization . . . 15
8.6.2 Mini-Batch Gradient Descent and Momentum . . . 16
8.6.3 Adam Optimizer . . . 16

9 Classification and Machine Learning 16
9.1 From Regression to Classification . . . 16
9.2 Evaluating a Binary Classifier . . . 16
9.2.1 The Confusion Matrix . . . 16
9.2.2 Metrics . . . 16

9.2.3 ROC Curve and AUC . . . 17
9.3 Loss Functions for Classification . . . 17
9.3.1 Why Not Accuracy? . . . 17
9.3.2 The Sigmoid (Logistic) Function . . . 17
9.3.3 Cross-Entropy Loss . . . 17
9.3.4 Regularization . . . 17
9.4 Multi-Class Classification . . . 17
9.4.1 Softmax: A Differentiable Argmax . . . 17
9.4.2 Temperature-Scaled Softmax . . . 18
9.5 Multi-Class Loss . . . 18
9.5.1 One-Hot Encoding and Mean Cross-Entropy . . . 18
9.5.2 Gradient via the Chain Rule . . . 18

10 Neural Networks **18**
10.1 Motivation: Beyond Linear Models . . . 18
10.2 Neural Network Architecture . . . 18
10.2.1 The Basic Building Block . . . 18
10.2.2 Batch Processing . . . 19
10.2.3 Universal Approximation Theorem . . . 19
10.3 Activation Functions . . . 19
10.3.1 Sigmoid . . . 19
10.3.2 ReLU (Rectified Linear Unit) . . . 19
10.3.3 Other Common Activations . . . 19
10.4 Training Neural Networks: Backpropagation . . . 20
10.4.1 Overview . . . 20
10.4.2 The Jacobian and Chain Rule . . . 20
10.4.3 Gradients for a Fully-Connected Layer . . . 20
10.4.4 Gradient Through ReLU . . . 20
10.4.5 Output Gradient: Softmax + Cross-Entropy . . . 20
10.5 Training in Practice . . . 20

11 Convolutional Neural Networks and Flax NNX **21**
11.1 Motivation: Fully-Connected Layers Don't Scale . . . 21
11.2 1D and 2D Convolution . . . 21
11.3 Dimension Reduction . . . 21
11.4 Typical CNN Architecture . . . 21
11.5 Flax NNX Code . . . 21
11.6 JAX Ecosystem . . . 22

12 Deep Learning **22**
12.1 Challenges in Deep Networks . . . 22
12.2 Data Augmentation . . . 22
12.3 Dropout . . . 22
12.4 Batch Normalization . . . 22
12.5 Residual Connections . . . 23
12.6 Transfer Learning . . . 23
12.7 Ethics of Deep Learning . . . 23

13 Recurrent Neural Networks **23**
13.1 Variable-Length Sequences . . . 23
13.2 Tokenization and Embeddings . . . 23
13.3 RNN Architecture . . . 23
13.4 Common RNN Configurations . . . 24
13.5 LSTM and GRU . . . 24
13.6 Attention and Transformers . . . 24

14 State/Action Search **24**
14.1 Problem Formulation . . . 24
14.2 Breadth-First Search (BFS) . . . 24
14.3 Depth-First Search (DFS) . . . 25
14.4 Iterative Deepening Depth-First Search (IDDFS) . . . 25
14.5 Best-First (Greedy) Search . . . 25
14.6 Bidirectional BFS . . . 25

Lecture 1: Probability for AI

1.1 Motivation

Robust AI systems must handle **uncertainty**. There are two interpretations:

- **Expectation**: "If I make this move, 10% of the time I win $110."
- **Certainty**: "I'm 42% certain that is a human in the road."

Systems with no room for uncertainty are fragile. Probability is the primary tool.

1.2 Probability Basics

A probability model has three parts:

1. **Sample space** Ω: all possible outcomes.
2. **Event** H: any subset of Ω.
3. **Probability** $P(H) \in \mathbb{R}$, where $P(H) = 0$ is impossible, $P(H) = 1$ is certain.

Axioms of Probability:

1. Non-negativity: $P(H) \geq 0$.
2. Normalization: $P(\Omega) = 1$.
3. Additivity: If events H_i are disjoint, $P(\bigcup_i H_i) = \sum_i P(H_i)$.

1.3 Joint, Marginal, and Conditional Probability

Joint probability distribution $P(X, Y)$ gives the probability over all combinations of two random variables.

Marginal distribution: Sum over all values of one variable to "ignore" it:

$$P(X = x) = \sum_{y \in Y} P(X = x, Y = y)$$

Conditional probability:

$$P(Y = y \mid X = x) = \frac{P(Y = y,\, X = x)}{P(X = x)}$$

1.4 Chain Rule for Probability

$$P(X, Y, Z) = P(X)\, P(Y \mid X)\, P(Z \mid X, Y)$$

This generalizes to any number of variables. It is the fundamental rule underlying modern AI.

1.5 Independence

X and Y are **independent** iff:

$$P(X = x, Y = y) = P(X = x)\, P(Y = y) \quad \forall\, x, y$$

Equivalently, $P(X = x \mid Y = y) = P(X = x)$ for all x, y.

1.6 Bayes' Law

Bayes' Law relates the **posterior**, **likelihood**, and **prior**:

$$P(A = a \mid B = b) = \frac{P(B = b \mid A = a)\, P(A = a)}{P(B = b)}$$

The denominator can be expanded: $P(B = b) = \sum_{a^*} P(B = b \mid A = a^*)\, P(A = a^*)$.

Compact form: **posterior** $\propto$ **likelihood** $\times$ **prior**.

Example (medical test): Disease prevalence = 0.1%. Test false positive rate = 1%, false negative = 0.5%. A positive result means only $\approx 9\%$ chance of actually having the disease. The prior (rare disease) dominates.

1.7 Continuous Random Variables

For continuous variables, replace probability mass functions with **probability density functions** (PDFs) $p(x)$, satisfying:

$$p(x) \geq 0, \quad \int_{-\infty}^{\infty} p(x)\, dx = 1$$

Key statistics:

$$\mu = E[X] = \int x\, p(x)\, dx, \quad \mathrm{Var}(X) = \int (x - \mu)^2 p(x)\, dx$$

Mean = center of mass; Mode = peak of curve; Median = divides area equally.

Normal (Gaussian) distribution:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

where μ is the mean and σ^2 is the variance.

1.8 Working in Log Space

Long products of small probabilities underflow. Use log space:

$$\log p(x_1, x_2, \ldots) = \log p(x_1) + \log p(x_2) + \cdots$$

To normalize, add a constant $c = -\max(\log a, \log b)$ before exponentiating to avoid numerical issues.

Lecture 2: Bayes' Rule with Continuous Random Variables

2.1 Continuous Bayes

When the unknown quantity is a real-valued parameter λ (the "box"), Bayes' rule becomes:

$$p(\lambda \mid \text{data}) \propto p(\text{data} \mid \lambda)\, p(\lambda), \quad \int p(\lambda \mid \text{data})\, d\lambda = 1$$

- $p(\lambda)$: **prior** – belief before seeing data
- $p(\text{data} \mid \lambda)$: **likelihood** – probability of the data given λ
- $p(\lambda \mid \text{data})$: **posterior** – updated belief

2.2 Exponential Distribution

For times between events: $T \sim \text{Exponential}(\lambda)$, where λ is the rate.

$$P(T \leq t) = 1 - e^{-\lambda t}, \quad p(T = t) = \lambda e^{-\lambda t}$$

Example: Estimating coffee-shop arrival rate λ from observed inter-arrival times.

2.3 Credible Intervals

A γ-**credible interval** contains γ of the posterior's probability mass. Start at the MAP (maximum a posteriori) estimate and expand left/right until γ is captured.

2.4 Three Approaches to Continuous Posteriors

1. **Symbolic / Conjugate Priors**: Some prior–likelihood pairs yield closed-form posteriors.
   - Gamma prior + exponential likelihood $\Rightarrow$ Gamma posterior.
   - Beta prior + Binomial likelihood $\Rightarrow$ Beta posterior.
2. **Numerical (Grid)**: Evaluate prior $\times$ likelihood on a grid, then normalize via numerical integration.

```
z = sum(posterior_unnorm) * delta
posterior = posterior_unnorm / z
```

3. **MCMC (Markov Chain Monte Carlo)**: For high-dimensional posteriors, use sampling methods such as NUTS via `NumPyro`.

```
import numpyro
from numpyro.infer import MCMC, NUTS
def model(asked, supporters):
    r = numpyro.sample("r", dist.Uniform(0, 1))
    numpyro.sample("obs", dist.Binomial(asked, r), obs=supporters)
```

2.5 Binomial Example: Polling

$$P(k) = \binom{n}{k} r^k (1-r)^{n-k}$$

Start with a Beta prior on r (e.g., $r \sim \text{Beta}(800, 200)$ if historically 80% support). After polling n people and seeing k supporters, the posterior is $\text{Beta}(800 + k, 200 + n - k)$. Repeated polls with $\sim 30\%$ support eventually overwhelm the strong prior.

Impact of Priors:

- Good priors speed convergence when data is limited.
- Bad priors can be overcome by enough data (as long as no event has prior probability exactly 0).

Lecture 3: Bayesian Networks

3.1 Motivation: Modeling Dependencies

Multiple variables can be related. Naive joint tables grow exponentially. Bayesian Networks encode conditional independence assumptions compactly.

3.2 Structure of a Bayesian Network

A Bayes Net is a **directed acyclic graph (DAG)** where:

- Each node is a random variable.
- Each node has a **conditional probability table (CPT)** conditioned on its parents.
- The joint distribution factors as:

$$P(X_1, \ldots, X_n) = \prod_i P(X_i \mid \text{parents}(X_i))$$

3.3 Conditional Independence

W and R are **conditionally independent given** A if:

$$P(W \mid A, R) = P(W \mid A) \quad (\text{written } W \perp\!\!\!\perp R \mid A)$$

The graph encodes exactly which independences hold.

3.4 Information Flow: Three Structures

Pipe (Chain) $A \to B \to C$**:** Information flows along the pipe unless B is observed.

Fork $A \leftarrow B \to C$**:** A and C are marginally dependent but independent given B.

Collider $A \to C \leftarrow B$**:** A and B are marginally independent; observing C *creates* dependence ("explaining away").

3.5 Inference in Bayes Nets

Given observations, update beliefs using Bayes' rule:

$$P(\text{query} \mid \text{evidence}) \propto P(\text{evidence} \mid \text{query})\, P(\text{query})$$

Methods for exact inference: Variable elimination, clique tree propagation.
Approximate inference: Sampling, loopy belief propagation.
Pruning: d-separation, Markov blankets.

3.6 Continuous Variables in Bayes Nets

CPTs are replaced by conditional probability distributions (e.g., Gaussians whose parameters depend on parent values).

Lecture 4: Bayesian and Naive Bayes Classifiers

4.1 Classification via Bayes' Rule

Given observations (features) $\mathbf{x}$, predict class C:

$$P(C \mid \mathbf{x}) \propto P(\mathbf{x} \mid C)\, P(C)$$

Normalize over all classes to get a proper distribution; predict the class with highest posterior.

4.2 Naive Bayes Classifier

Assumption: Features are conditionally independent given the class.

$$P(x_1, \ldots, x_d \mid C) = \prod_{j=1}^{d} P(x_j \mid C)$$

This is a "lie" in practice (features are often correlated), but classifiers work well anyway.

Steps:

1. Estimate **priors** $P(C)$ by counting class frequencies.
2. For each feature and class, fit a distribution (e.g., Gaussian) and compute likelihood.
3. Multiply likelihoods for joint likelihood; multiply by prior for unnormalized posterior.
4. Normalize and predict the highest-probability class.

4.3 Multivariate Gaussian Classifier

To model correlated features, use the **multivariate Gaussian**:

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma)}} \exp\left(-\tfrac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})\right)$$

Parameter estimation from n labeled samples:

$$\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{x}_i, \quad \Sigma = \frac{1}{n} \sum_{i=1}^{n} (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T$$

The covariance matrix Σ is symmetric; off-diagonal entries capture correlations between features.

4.4 Conditional Distribution of a Multivariate Gaussian

Given $\mathbf{x} = (x_1, x_2)^T \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ and observing x_2:

$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2), \quad \Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

4.5 Parameterless Density Estimation

When data doesn't fit a known distribution, use **Kernel Density Estimation (KDE)**:

```python
from scipy.stats import gaussian_kde
density = gaussian_kde(data)
print(density(6.0))   # density at point 6.0
```

4.6 Summary of Classifier Types

Method	Joint Likelihood Computation
Naive Bayes	Product of univariate distributions (independence assumed)
True Bayes (parametric)	Multivariate distribution (e.g., multivariate Gaussian)
True Bayes (nonparametric)	Kernel density estimator

Lecture 5: Markov Chains and Hidden Markov Models

5.1 Markov Chains

A **Markov chain** models a sequence of random variables $X_0, X_1, X_2, \ldots$ with the **Markov assumption**:

$$X_{i+1} \perp\!\!\!\perp X_j \mid X_i \quad \forall j < i$$

So: $P(X_0, X_1, \ldots, X_n) = P(X_0) \prod_{t=1}^{n} P(X_t \mid X_{t-1})$

Defined by:

- **Initial distribution** $\pi = P(X_0)$
- **Transition matrix** T where $T_{ij} = P(X_t = j \mid X_{t-1} = i)$

5.2 Mini-Forward Algorithm

To propagate the distribution forward one step:

$$P(X_t) = P(X_{t-1})\, T$$

5.3 Stationary Distribution

As $t \to \infty$, an **ergodic** chain converges to a unique stationary distribution π_∞ satisfying:

$$\pi_\infty = \pi_\infty T$$

π_∞ is the left eigenvector of T with eigenvalue 1. Compute in Python:

```
vals, vecs = jnp.linalg.eig(T.T)
idx = jnp.argmin(jnp.abs(vals - 1.0))
pi = vecs[:, idx].real
pi = pi / pi.sum()
```

5.4 Hidden Markov Models (HMMs)

HMMs extend Markov chains with **emissions**: at each time step, an observation Y_t is generated from the hidden state X_t.

- π: initial state distribution
- T: transition matrix
- $E_{i,j}$: emission matrix, $P(Y_t = j \mid X_t = i)$
- X_t is **unobserved**; only Y_t is seen

5.5 Key HMM Algorithms

Filtering (Forward Algorithm): Given π, T, E and observations $y_1, \ldots, y_t$, compute $P(X_t \mid y_1, \ldots, y_t)$.

Smoothing (Forward-Backward): Compute $P(X_k \mid y_1, \ldots, y_t)$ for $k < t$.

Decoding (Viterbi): Find the most likely state sequence $\hat{x}_1, \ldots, \hat{x}_t$ given observations. Uses dynamic programming with matrices J (max-probability) and Q (backpointer):

$$J_{x,0} = \pi_x \cdot E_{x,y_0}$$
$$J_{x,t} = \max_{x'} J_{x',t-1} \cdot T_{x'x} \cdot E_{x,y_t}$$

Trace back using Q to recover the optimal path.

Learning (Baum-Welch): Estimate π, T, E from unlabeled observations (EM algorithm).

5.6 Application: Speech Recognition

- **Training**: Convert audio to Mel spectrogram frames, quantize, use Baum-Welch to learn π_w, T_w, E_w for each word w.
- **Decoding**: Quantize new audio, use Viterbi + language model prior to select most likely word.

5.7 Google PageRank

A Markov chain over web pages: with probability c jump to a random page; with probability $1 - c$ follow a random link. The stationary distribution gives page rankings.

Lecture 6: Kalman Filters and Particle Filters

6.1 The Localization Problem

Robots must estimate position (x, y, θ) from noisy sensors and noisy actuators. Two elegant solutions: **Particle Filters** (nonparametric) and **Kalman Filters** (parametric, linear-Gaussian).

6.2 Particle Filters

Represent the belief state as a set of n weighted samples ("particles"):

1. **Initialize**: Sample n particles $\{(x_i, y_i, \theta_i)\}$ uniformly over the map.
2. **Observe**: Get sensor measurements.
3. **Weight**: Compute likelihood $w_i = P(\text{measurements} \mid \text{particle}_i)$.
4. **Resample**: Draw n new particles with replacement, weighted by w_i. Unlikely particles are culled; likely ones multiply.
5. **Roughen**: Add noise to particles to avoid degeneracy.
6. **Act**: Simulate the action on all particles.

7. Repeat from step 2.

Sensor noise modeled as a Gaussian: if predicted distance is d and measured is d', likelihood $\propto \mathcal{N}(d' - d;\, 0,\, \sigma^2)$.

6.3 Kalman Filters

Developed by Rudolf Kálmán (1960); used in Apollo missions and all modern aerospace.

State model (linear system with Gaussian noise):

$$\mathbf{x}_t = \mathbf{F}\,\mathbf{x}_{t-1} + \mathbf{B}\,\mathbf{u}_{t-1} + \mathbf{w}, \quad \mathbf{w} \sim \mathcal{N}(0, \mathbf{Q})$$

Observation model:

$$\mathbf{z}_t = \mathbf{H}\,\mathbf{x}_t + \mathbf{v}, \quad \mathbf{v} \sim \mathcal{N}(0, \mathbf{R})$$

Matrices:

- $\mathbf{F}$: state transition (physics)
- $\mathbf{B}$: control input effect
- $\mathbf{Q}$: process noise covariance
- $\mathbf{H}$: observation model (maps state to sensor space)
- $\mathbf{R}$: sensor noise covariance

6.4 Kalman Filter Loop

1. **Predict state and error covariance**:

$$\hat{\mathbf{x}}_t = \mathbf{F}\hat{\mathbf{x}}_{t-1} + \mathbf{B}\mathbf{u}_{t-1}, \quad \mathbf{P}_t = \mathbf{F}\mathbf{P}_{t-1}\mathbf{F}^T + \mathbf{Q}$$

2. **Compute residual**: $\tilde{\mathbf{y}} = \mathbf{z}_t - \mathbf{H}\hat{\mathbf{x}}_t$
3. **Compute Kalman Gain**:

$$\mathbf{S} = \mathbf{H}\mathbf{P}_t\mathbf{H}^T + \mathbf{R}, \quad \mathbf{K} = \mathbf{P}_t\mathbf{H}^T\mathbf{S}^{-1}$$

   $\mathbf{K}$ entries near 1 = trust sensor; near 0 = trust prediction.

4. **Update state and covariance**:

$$\hat{\mathbf{x}}_t \leftarrow \hat{\mathbf{x}}_t + \mathbf{K}\tilde{\mathbf{y}}, \quad \mathbf{P}_t \leftarrow (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}_t$$

Extensions: Extended Kalman Filter (EKF) for differentiable nonlinear f, h; Unscented Kalman Filter (UKF) for highly nonlinear systems.

Lecture 7: Intro to ML, K-Nearest Neighbors, and Decision Trees

7.1 Flavors of Machine Learning

Supervised Labeled data. *Classification* (discrete output) or *Regression* (continuous output).

Unsupervised No labels. *Clustering.*

Self-Supervised / Generative Learning structure from unlabeled data; generate new examples.

7.2 ML Workflow

1. Define the goal.
2. Gather and label data. Split into **train** and **test** sets.
3. Choose a model and hyperparameters.
4. Train on training set.
5. Evaluate on test set.
6. Deploy.

Diagnosing problems:

- Good train, bad test ⇒ **overfitting** (high variance).
- Bad train, bad test ⇒ **underfitting** (high bias).

7.3 K-Nearest Neighbors (kNN)

1-NN: Find the single closest training point; return its label. Generalization: find the k closest points and take a majority vote.

Common distance metrics:

$$d_2(\mathbf{a}, \mathbf{c}) = \sqrt{\sum_i (a_i - c_i)^2} \quad \text{(Euclidean)}$$

$$d_1(\mathbf{a}, \mathbf{c}) = \sum_i |a_i - c_i| \quad \text{(Manhattan)}$$

In high dimensions both are poorly behaved; prefer **cosine similarity**:

$$\text{sim}(\mathbf{a}, \mathbf{c}) = \frac{\mathbf{a} \cdot \mathbf{c}}{|\mathbf{a}||\mathbf{c}|}$$

Hyperparameters: k, distance metric, weighting function.

7.4 Validation and Cross-Validation

Hold out a **validation set** to tune hyperparameters. Use n**-fold cross-validation**: partition training data into n folds; train on $n-1$ and validate on 1; rotate and average accuracy.

7.5 Common Problems and Fixes

Different scales Standardize: $z = (x - \mu)/\sigma$.

Garbage features Feature selection; remove uncorrelated features.

Unbalanced samples Resample or reweight classes.

Slow / bloated Use **Approximate Nearest Neighbor** (ANN) libraries: FAISS, ScaNN, Annoy.

7.6 Decision Trees

Split the feature space recursively to minimize **impurity**. At each node, choose the feature and threshold that best separates classes.

Gini Impurity for a node with class proportions p_k:

$$G = 1 - \sum_k p_k^2$$

Alternative criteria: Entropy, Log Loss (all behave similarly; Gini is fastest).

Splitting continuous features: Try all midpoints between adjacent sorted values; pick the one with best impurity reduction.

Hyperparameter tuning: `max_depth`, `min_samples_split`, `max_leaf_nodes` — pruning prevents overfitting.

7.7 Ensemble Methods

Random Forest: Train ∼1000 trees, each on a random subset of $\sqrt{d}$ features and a bootstrap sample. Aggregate predictions by majority vote (classification) or average (regression).

Gradient Boosted Trees (e.g., XGBoost): Build trees sequentially; each new tree corrects the residual errors of the combined ensemble.

Lecture 8: Linear Regression and Gradient Descent

8.1 The Machine Learning Workflow

Every machine learning problem follows the same high-level pattern:

1. **Model** — Develop a model with a set of tunable parameters.
2. **Loss Function** — Define a function that measures how well the model fits the data. Lower is better.
3. **Train** — Use data to optimize the parameters so that the loss function is minimized.
4. **Test** — Evaluate the trained model on unseen data to measure real-world performance.
5. **Deploy** — Put the model into production.

> This workflow is universal. Whether you are fitting a two-parameter line or training a trillion-parameter language model, these five steps always apply.

8.2 Simple Linear Regression

8.2.1 The Model

Consider predicting how much power (watts) can be extracted from gasoline based on its water content. We hypothesize a linear relationship:

$$\hat{y} = mx + b$$

where x is the percentage of water in the fuel and $\hat{y}$ is the predicted power output. The **parameters** are the slope m and the intercept b.

8.2.2 The Bias-Variance Tradeoff

> GPT-4 has approximately 1.76 trillion parameters. Our model has 2.

The number of parameters is a fundamental design choice:

- **Too many parameters** (high variance / overfitting): The model memorizes the training data and performs poorly on unseen data.
- **Too few parameters** (high bias / underfitting): The model is not expressive enough to capture the true patterns.

8.3 Loss Functions

8.3.1 L1 Loss (Mean Absolute Error)

$$\mathcal{L}_1 = \sum_i |y_i - \hat{y}_i|$$

Problem: L1 loss is not unique — multiple very different lines can achieve the same L1 loss, making optimization unreliable.

> *Picking the wrong loss function can doom your efforts.*

8.3.2 L2 Loss (Mean Squared Error)

Squaring the errors fixes the uniqueness problem and penalizes large errors more heavily:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$$

Using the mean (dividing by n) instead of the sum ensures the loss magnitude does not depend on batch size.

8.4 Solving Linear Regression with Matrices

For n training examples with d features, augment inputs with a bias column:

$$\boldsymbol{X}_{n\times(d+1)}, \quad \vec{y}_{n\times 1}, \quad \vec{\beta}_{(d+1)\times 1}$$

Predictions are $\hat{y} = \boldsymbol{X}\vec{\beta}$, and the L2 loss is $\mathcal{L}(\vec{\beta}) = \vec{r}^T\vec{r}$ where $\vec{r} = \vec{y} - \boldsymbol{X}\vec{\beta}$.

Taking the gradient and setting to zero gives the **Normal Equations**:

$$\boxed{\vec{\beta} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\vec{y}}$$

This closed-form solution is exact and optimal because the L2 loss for linear regression is convex.

> **Limitation:** Neural networks have no closed-form solution — their loss is non-convex and high-dimensional. We must use **Gradient Descent**.

8.5 Gradient Descent

8.5.1 The Gradient

For $f(\vec{x}) : \mathbb{R}^n \to \mathbb{R}$, the gradient ∇f points in the direction of steepest ascent. Therefore $-\nabla f$ points in the direction of steepest **descent**.

8.5.2 The Algorithm

1. Initialize parameters θ_0 (often random).
2. Compute gradient $\nabla\mathcal{L}(\theta_t)$.
3. Update: $\theta_{t+1} := \theta_t - \alpha\,\nabla\mathcal{L}(\theta_t)$ ($\alpha > 0$ is the **learning rate**).
4. Repeat until $\|\nabla\mathcal{L}(\theta_t)\|$ is sufficiently small.

For linear regression with MSE:

$$\nabla\mathcal{L}(\beta) = \frac{2}{n}\boldsymbol{X}^T\left(\boldsymbol{X}\vec{\beta} - \vec{y}\right)$$

8.5.3 Choosing the Learning Rate

- **Too large:** Overshoots the minimum; loss oscillates or diverges.
- **Too small:** Extremely slow convergence.

Gradually decrease α over training (**learning rate scheduling**): start large for fast progress, shrink to fine-tune.

8.5.4 Local Minima in High Dimensions

For a stable local minimum to exist, *every* second partial derivative must be positive simultaneously — extremely unlikely in millions of dimensions. Most apparent local minima in deep learning are actually **saddle points** from which gradient descent naturally escapes.

8.6 Practical Improvements to Gradient Descent

8.6.1 Data Standardization

When features have different scales, the loss landscape is ill-conditioned. Fix: standardize each feature to zero mean, unit variance:

$$\hat{x}_i = \frac{x_i - \mu_i}{\sigma_i}$$

Critical rule: Compute μ and σ on the *training* set only. Apply those same values at test time — never recompute on test data (data leakage).

8.6.2 Mini-Batch Gradient Descent and Momentum

Split large datasets into **mini-batches**. Noisy gradients are smoothed with **momentum**: an exponential running average of past gradients:

$$\vec{v} := m\vec{v} + (1-m)\,\nabla\mathcal{L}(\theta_i), \qquad \theta_{i+1} := \theta_i - \alpha\vec{v}$$

where $m \in [0, 1)$ controls velocity retention. Momentum also helps escape saddle points.

8.6.3 Adam Optimizer

Adam (Adaptive Moment Estimation, Kingma & Ba 2014) is the default optimizer in modern deep learning. It combines:

- **Momentum**: running average of gradients (1st moment).
- **Adaptive learning rates**: scales per-parameter using running average of squared gradients (2nd moment).
- **Bias correction**: corrects for zero-initialization of moment estimates.

Central theme: *Differentiable = Good.* Gradient descent can only optimize functions it can differentiate. Every design choice in modern ML — activations, loss functions, softmax — is driven by the requirement of differentiability.

Lecture 9: Classification and Machine Learning

9.1 From Regression to Classification

The same ML workflow applies to classification. The challenge is adapting each piece for discrete class-label outputs. Key requirement: a *differentiable* loss function over the parameters.

9.2 Evaluating a Binary Classifier

9.2.1 The Confusion Matrix

	Predicted Negative	**Predicted Positive**
Actually Negative	True Negative (TN)	False Positive (FP)
Actually Positive	False Negative (FN)	True Positive (TP)

9.2.2 Metrics

$$\text{Accuracy} = \frac{\text{TP}+\text{TN}}{n}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP}+\text{FN}}, \quad \text{Precision} = \frac{\text{TP}}{\text{TP}+\text{FP}}$$

$$\text{F1} = 2 \times \frac{\text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}}$$

Recall vs. Precision tradeoff: In medical diagnosis, missing a true positive (low recall) can be fatal — lower the threshold. In spam filtering, falsely flagging legitimate emails (low precision) is costly — raise the threshold. The threshold is a business decision, not a model decision.

9.2.3 ROC Curve and AUC

A **ROC** curve sweeps the decision threshold from 0 to 1, plotting True Positive Rate vs. False Positive Rate at each point. The **AUC** (Area Under the Curve) summarizes this in a single number: 1.0 = perfect, 0.5 = random. AUC is threshold-independent and is often the best single metric for comparing classifiers.

9.3 Loss Functions for Classification

9.3.1 Why Not Accuracy?

Accuracy, recall, and F1 depend on a hard threshold and are **not differentiable**. We need smooth probability-like outputs.

9.3.2 The Sigmoid (Logistic) Function

$$\sigma(x) = \frac{1}{1+e^{-x}} \in (0,1), \qquad \sigma'(x) = \sigma(x)(1-\sigma(x))$$

Composing sigmoid with a linear model: $\hat{y} = \sigma(\beta^T x) \in (0,1)$.

9.3.3 Cross-Entropy Loss

Derived from maximum likelihood estimation; convex and differentiable:

$$\boxed{\mathrm{CE} = -\frac{1}{n}\sum_i \big[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\big]}$$

Logistic Regression: linear features → sigmoid → optimize with cross-entropy. Despite the name, it is a *classifier*.

9.3.4 Regularization

If training data is linearly separable, cross-entropy pushes weights to $\pm\infty$. Add L2 regularization to prevent this:

$$\mathcal{L}(\beta) = \mathrm{CE}(\beta) + \lambda\, \beta^T \beta$$

9.4 Multi-Class Classification

9.4.1 Softmax: A Differentiable Argmax

For k classes, assign each class a score $z_c = \vec{\beta}_c^T \vec{x}$. The **softmax** converts scores to probabilities:

$$\mathrm{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Properties: outputs in $(0, 1)$, sum to 1, differentiable everywhere. It is the gradient of the smooth $\log \sum_k e^{x_k}$ approximation to max.

9.4.2 Temperature-Scaled Softmax

$$\text{softmax}_T(r_i) = \frac{\exp(r_i/T)}{\sum_j \exp(r_j/T)}$$

$T \to 0$: one-hot (confident); $T = 1$: standard; $T \to \infty$: uniform (uncertain). Used in knowledge distillation and controlling language model output randomness.

9.5 Multi-Class Loss

9.5.1 One-Hot Encoding and Mean Cross-Entropy

With n samples and k classes, let $\boldsymbol{Y}$ be the $n \times k$ one-hot matrix and $\hat{\boldsymbol{Y}}$ the softmax outputs:

$$\boxed{\mathcal{L} = -\sum_{i=1}^{n} \sum_{j=1}^{k} y_{i,j} \log(\hat{y}_{i,j})}$$

Only the log-probability of the correct class contributes per sample (since $y_{i,j}$ is one-hot).

9.5.2 Gradient via the Chain Rule

The combined gradient of softmax + cross-entropy w.r.t. pre-softmax scores $\boldsymbol{Z} = \boldsymbol{X\beta}$ is elegantly simple:

$$\frac{\partial \mathcal{L}}{\partial z_{i,j}} = \hat{y}_{i,j} - y_{i,j} \qquad \Longrightarrow \qquad \boxed{\nabla_\beta \mathcal{L} = \boldsymbol{X}^T(\hat{\boldsymbol{Y}} - \boldsymbol{Y})}$$

Prediction minus truth. Elegant and computationally cheap.

Lecture 10: Neural Networks

10.1 Motivation: Beyond Linear Models

Linear models only model linear relationships. Real-world data almost never is linearly separable. We need a model that can approximate *any* function $f(\vec{x}) \to \mathbb{R}$.

Intuition: Think of non-linear data as a crumpled piece of paper. A linear model tries to cut through it with a flat plane — hopeless. A neural network can *bend* the space to flatten it first, then apply a linear separator.

10.2 Neural Network Architecture

10.2.1 The Basic Building Block

Each neuron in a hidden layer: (1) takes a *weighted sum* of all inputs plus a bias; (2) passes through a non-linear **activation function**.

For inputs $\vec{x}$, hidden layer weights W^1, biases $\vec{b}^1$, and activation σ:

$$\vec{a} := \sigma\left(\vec{x}\,W^1 + \vec{b}^1\right)$$

Without the activation, stacking layers is still just one linear map. Activations introduce the non-linearity that makes neural networks powerful.

10.2.2 Batch Processing

For a batch of n examples with d inputs and k hidden units:

$$Y_{n\times k} := X_{n\times d} \cdot W_{d\times k} + B_{n\times k}$$

Processing batches is critical for GPU efficiency — matrix-matrix multiplication is far faster than repeated matrix-vector multiplication.

10.2.3 Universal Approximation Theorem

Theorem 1 (Universal Approximation, 1989)**.** *A neural network with a single hidden layer of sufficient width and a sigmoid activation can approximate any continuous function to arbitrary precision.*

In practice, depth beats width: deeper networks learn better representations with fewer total parameters.

10.3 Activation Functions

10.3.1 Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}, \qquad \sigma'(x) = \sigma(x)(1-\sigma(x))$$

Natural for probabilities. **Downside:** saturates for large $|x|$, causing vanishing gradients.

10.3.2 ReLU (Rectified Linear Unit)

$$\mathrm{relu}(x) = \max(0, x), \qquad \mathrm{relu}'(x) = \begin{cases} 0 & x < 0 \\ 1 & x > 0 \end{cases}$$

The **default** for hidden layers. Gradient is 0 or 1 — greatly reduces vanishing gradients; cheap to compute.

> **Dying ReLU:** If a neuron's pre-activation is always negative, its gradient is always 0 and it never learns. Leaky-ReLU fixes this with a small slope $\alpha > 0$ for $x < 0$.

10.3.3 Other Common Activations

Tanh: $\tanh(x) \in (-1, 1)$; zero-centered, often better than sigmoid for hidden layers.

Leaky-ReLU: $\max(\alpha x, x)$ with small α; prevents dying neurons.

Swish: $x \cdot \sigma(x)$; smooth, self-gated; often outperforms ReLU in deep networks.

10.4 Training Neural Networks: Backpropagation

10.4.1 Overview

Training alternates between two passes:

- **Forward pass:** Feed data through network; compute and store intermediate activations; compute loss.
- **Backward pass:** Walk backwards through network; compute gradients via the chain rule; update parameters.

10.4.2 The Jacobian and Chain Rule

For $f : \mathbb{R}^d \to \mathbb{R}^m$, the **Jacobian** J_f has $(J_f)_{i,k} = \partial f_i / \partial x_k$. Chain rule in matrix form: if $h : \mathbb{R}^m \to \mathbb{R}$, then $\nabla[h \circ f] = J_f^T(\nabla h)$.

10.4.3 Gradients for a Fully-Connected Layer

For a layer $Z_{\text{out}} = Z_{\text{in}} W + B$ followed by loss $\mathcal{L}$, given upstream gradient $\nabla_{Z_{\text{out}}} \mathcal{L}$:

Weights: $\nabla_W \mathcal{L} = Z_{\text{in}}^T [\nabla_{Z_{\text{out}}} \mathcal{L}]$

Biases: $\nabla_{\vec{b}} \mathcal{L} =$ column sum of $\nabla_{Z_{\text{out}}} \mathcal{L}$

Upstream: $\nabla_{Z_{\text{in}}} \mathcal{L} = [\nabla_{Z_{\text{out}}} \mathcal{L}]\, W^T$

10.4.4 Gradient Through ReLU

ReLU is applied element-wise; its gradient is a simple **mask**: pass the upstream gradient through wherever the input was positive, zero it out elsewhere.

$$\frac{\partial \mathcal{L}}{\partial Z1_{i,j}} = \begin{cases} \nabla_{Z2} \mathcal{L}_{i,j} & Z1_{i,j} \geq 0 \\ 0 & Z1_{i,j} < 0 \end{cases}$$

10.4.5 Output Gradient: Softmax + Cross-Entropy

The combined gradient of softmax + mean cross-entropy w.r.t. the pre-softmax scores is:

$$\nabla_{Z_{\text{out}}} \mathcal{L} = \frac{1}{n}(\hat{y} - y)$$

Prediction minus truth — the cleanest result in all of backpropagation.

10.5 Training in Practice

A healthy training run shows loss decreasing steeply at first, then leveling off. Key sanity checks:

- Loss should decrease; if it increases or produces NaNs, there is likely a gradient bug.
- Decreasing cross-entropy should correspond to improving classification accuracy.
- Test accuracy matters: if training loss drops but test accuracy stagnates, you are overfitting.

Lecture 11: Convolutional Neural Networks and Flax NNX

11.1 Motivation: Fully-Connected Layers Don't Scale

A 1080p image has $1920 \times 1080 \times 3 \approx 6.2$M inputs. A single FC layer with 1000 units needs ~ 6.2B parameters. This is infeasible.

11.2 1D and 2D Convolution

A **convolution** slides a small **kernel** over the input, computing a dot product at each location:

$$(\text{output})_{i,j} = \sum_{m,n} \text{kernel}_{m,n} \cdot \text{input}_{i+m,\, j+n}$$

Zero-padding keeps spatial dimensions constant. **Stride** S subsamples the output.

Output size with input W, kernel K, stride S, padding P:

$$\text{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

A color image has 3 channels; a kernel has shape $(\text{in_channels}, H, W)$. Multiple kernels produce multiple output channels (feature maps) with very few parameters compared to fully connected.

11.3 Dimension Reduction

Stride > 1: Every S-th position is kept, reducing spatial size.

Max Pooling: Divide into non-overlapping windows; keep the maximum in each. Reduces size by pool factor per channel independently.

Standard order: **Convolution** $\rightarrow$ **ReLU** $\rightarrow$ **Max Pool**.

11.4 Typical CNN Architecture

$$\underbrace{\text{Conv-ReLU-Pool}}_{\text{tall, wide, thin}} \longrightarrow \cdots \longrightarrow \underbrace{\text{Flatten}}_{\text{1D}} \longrightarrow \underbrace{\text{FC-Softmax}}_{\text{classification}}$$

11.5 Flax NNX Code

```python
from flax import nnx
import optax

class MNISTModel(nnx.Module):
    def __init__(self, *, rngs):
        self.fc1 = nnx.Linear(784, 128, rngs=rngs)
        self.fc2 = nnx.Linear(128, 10, rngs=rngs)
    def __call__(self, x):
        return nnx.softmax(self.fc2(nnx.relu(self.fc1(x))))

model = MNISTModel(rngs=nnx.Rngs(0))
optimizer = nnx.Optimizer(model, optax.adamw(1e-3), wrt=nnx.Param)
```

```python

@nnx.jit
def loss_fn(model, X, y):
    return optax.softmax_cross_entropy_with_integer_labels(
        logits=model(X), labels=y).mean()

grad_fn = nnx.value_and_grad(loss_fn)
for epoch in range(EPOCHS):
    loss, grads = grad_fn(model, batch_X, batch_y)
    optimizer.update(model, grads)
```

11.6 JAX Ecosystem

- **JAX**: NumPy + autodiff + JIT compilation.
- **Flax NNX**: Neural network layers.
- **Optax**: Optimizers (Adam, AdamW, SGD, etc.).
- Used by Anthropic, DeepMind, Midjourney, and others.

Lecture 12: Deep Learning

12.1 Challenges in Deep Networks

Problem	Solution
Overfitting	Data augmentation, Dropout
Vanishing / Exploding gradients	Batch normalization, Residual connections
Information loss	Residual / skip connections
Slow training	Transfer learning, better optimizers

12.2 Data Augmentation

Artificially expand limited training datasets by applying random transformations: flips, rotations, crops, color jitter. Forces the model to learn invariant features.

12.3 Dropout

During training, randomly zero out a fraction p of activations. Non-zeroed outputs are scaled by $\frac{1}{1-p}$ to maintain expected value. At inference, dropout is disabled.

```python
m = nn.Dropout(p=0.2)
```

Prevents co-adaptation of neurons; acts as implicit model averaging.

12.4 Batch Normalization

For an input batch with m samples and dimension d, normalize each feature k:

$$\hat{x}_i^k = \frac{x_i^k - \mu^k}{\sigma^k}, \quad y_i^k = g^k \hat{x}_i^k + b^k$$

where g^k and b^k are learned parameters. Applied after each layer to counteract distributional shift from nonlinearities.

12.5 Residual Connections

Introduced in ResNet (2015, won ImageNet):

$$\text{output} = F(\mathbf{x}) + \mathbf{x}$$

Skip connections bypass layers, providing identity gradients that prevent vanishing gradient and preserve high-frequency information. Present in all Transformers.

DenseNets: Instead of adding, concatenate feature maps from all previous layers.

12.6 Transfer Learning

Start from a large pre-trained model. Fine-tune only the final classification layers:

```
classifier_params = nnx.All(nnx.Param,
    nnx.PathContains('classifier'))
optimizer = nnx.Optimizer(model, optax.adamw(3e-4),
    wrt=classifier_params)
```

Pre-trained feature extractors generalize well; only the task-specific head needs retraining.

12.7 Ethics of Deep Learning

Key concerns: computers impersonating people, job displacement, wealth concentration, bias, overselling capabilities, energy consumption, hardware supply chains, privacy.

Lecture 13: Recurrent Neural Networks

13.1 Variable-Length Sequences

Standard MLPs and CNNs have fixed input/output sizes. Sequences (text, time series, audio) have variable length. Solution: **Recurrent Neural Networks (RNNs)**.

13.2 Tokenization and Embeddings

Words are first mapped to tokens (integers), then to dense vectors via an **embedding layer**.

- **Word2Vec** (2013): 100–300-dimensional embeddings; synonyms cluster together.
- Modern models use learned subword tokenizers (e.g., BPE with ∼100K vocab in GPT-4).

13.3 RNN Architecture

An RNN processes one element at a time, maintaining a **hidden state** passed from step to step:

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

Effectively an extremely deep network when "unrolled" over time. Trained with **Backpropagation Through Time (BPTT)**.

13.4 Common RNN Configurations

Many-to-one: Sentiment classification (sequence → label).

One-to-many: Image captioning (vector → sequence).

Many-to-many (same length): Per-step stock prediction.

Many-to-many (different length): Machine translation (encoder-decoder).

13.5 LSTM and GRU

Plain RNNs suffer from vanishing/exploding gradients over long sequences. **Long Short-Term Memory (LSTM)** and **Gated Recurrent Unit (GRU)** address this with **gating mechanisms** that control information flow rather than multiplying through many layers.

- LSTM maintains a **cell state** c_t (long-term memory) and a hidden state h_t (short-term).
- Gates: forget, input, output; each is a sigmoid applied to a linear combination.

```
from flax.nnx.nn.recurrent import RNN, LSTMCell
c = LSTMCell(in_features=100, hidden_features=30, rngs=rngs)
self.rnn = RNN(c)
```

13.6 Attention and Transformers

RNNs still lose information over long sequences (the "telephone game"). The **Transformer** architecture (Vaswani et al., 2017, "Attention Is All You Need") introduced **self-attention**, allowing every position to directly attend to every other position. This has become the dominant architecture for NLP (GPT, BERT) and is being applied across modalities.

Lecture 14: State/Action Search

14.1 Problem Formulation

A search problem is defined by:

- Initial state s_0
- Goal test: $\text{Goal}(s) \to \text{bool}$
- Action set: $A(s)$ = available actions in state s
- Transition function: $T(a, s) \to s'$

The **state space** is all reachable states. Two key questions: Does a solution exist? What is the shortest solution?

14.2 Breadth-First Search (BFS)

Uses a FIFO queue. Explores all nodes at depth d before depth $d + 1$.

Properties:

- Complete: Yes (finds solution if one exists).

- Optimal: Yes (fewest steps).
- Time and space: $O(b^x)$ where b = branching factor, x = depth of shallowest solution.

Check-before-push optimization: check the goal when pushing children, not when popping, to save one full level of memory.

Handling graphs (cycles): Maintain a **backtracking table** (map from state to parent action and state). Serves dual purpose: cycle detection and path reconstruction.

```
q = Queue(); bt = {}
q.push(s0); bt[s0] = (None, None)
while q:
    s = q.pop()
    for a in A(s):
        s2 = T(a, s)
        if s2 in bt: continue
        bt[s2] = (a, s)
        if Goal(s2): return backtrack(s0, s2, bt)
        q.push(s2)
```

BFS invariant: When the node at the front of the queue is n steps from s_0, every state within n steps is already in the backtracking table.

14.3 Depth-First Search (DFS)

Uses a LIFO stack.

- **Not** guaranteed to find shortest path.
- May not terminate on infinite trees.
- Space: $O(mb)$ where m = depth limit — much better than BFS.

14.4 Iterative Deepening Depth-First Search (IDDFS)

Run DFS with depth limits $1, 2, 3, \ldots$ until a solution is found. Achieves BFS-optimal solution with DFS-level $O(mb)$ space. Less wasteful than it appears; the last level dominates cost.

14.5 Best-First (Greedy) Search

Uses a priority queue ordered by a heuristic $h(s) \approx$ estimated distance to goal. Finds solutions much faster in practice but is not guaranteed to be optimal.

14.6 Bidirectional BFS

Search simultaneously from the initial state (forward) and goal state (backward). Meet in the middle.

- Naive version: Stop when the two frontiers meet. **Problem**: may not find the shortest path at first contact.
- **Correct termination**: Let μ = length of best solution found so far, n_f = depth of forward frontier, n_b = depth of backward frontier. Stop when $n_f + n_b \geq \mu$. Complexity: $O(b^{n/2})$ vs $O(b^n)$ for unidirectional.